

JAVA

BASICS MATH

15 Core Number & Mathematical Concepts
with Logic • Java Code • Examples • Complexity • Common Mistakes

From beginner arithmetic → problem-solving patterns

How to use these notes

Learn the idea first, then trace the example by hand, then type the Java code yourself. For DSA, these number operations become building blocks for arrays, hashing, recursion, mathematics, and competitive programming.

Contents

No.	Topic
01	Even & Odd Numbers
02	Count Number of Digits
03	Extract Last Digit
04	Remove Last Digit
05	Reverse a Number
06	Prime Number
07	Count Factors
08	GCD / HCF
09	LCM
10	Armstrong Number
11	Palindrome Number
12	Powers / Exponents
13	Square Root Optimization
14	Modulus Operator
15	Logarithms

Golden rule

Most basic-number problems can be solved by combining just four operations: % 10 to get the last digit, / 10 to remove the last digit, a loop to repeat the process, and a condition to make a decision.

Even & Odd Numbers

An integer is even when it is divisible by 2 with remainder 0. Otherwise it is odd.

KEY FORMULA / RULE

$n \% 2 == 0 \rightarrow \text{EVEN}$

$n \% 2 != 0 \rightarrow \text{ODD}$

Java implementation

```
import java.util.Scanner;

public class EvenOdd {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        if (n % 2 == 0) {
            System.out.println("Even");
        } else {
            System.out.println("Odd");
        }
    }
}
```

Worked example

For $n = 18$, $18 \% 2 = 0 \rightarrow \text{Even}$. For $n = 17$, $17 \% 2 = 1 \rightarrow \text{Odd}$.

COMPLEXITY

Time: $O(1)$ Space: $O(1)$

COMMON MISTAKE / TIP

Do not use $n / 2 == 0$. Division tells you the quotient; modulus tells you the remainder.

Count Number of Digits

Repeatedly remove the last digit using integer division by 10. Every successful division corresponds to one digit removed.

KEY FORMULA / RULE

$\text{digits} = \text{floor}(\log_{10}(n)) + 1$ (for $n > 0$)

Java implementation

```
public class CountDigits {  
    static int countDigits(int n) {  
        if (n == 0) return 1;  
  
        int count = 0;  
        n = Math.abs(n);  
  
        while (n > 0) {  
            n = n / 10;  
            count++;  
        }  
        return count;  
    }  
}
```

Worked example

12345 → 1234 → 123 → 12 → 1 → 0. The loop runs 5 times, so the answer is 5.

COMPLEXITY

Time: $O(\log_{10} n)$ Space: $O(1)$

COMMON MISTAKE / TIP

Handle 0 separately if you use the division loop, because 0 itself has one digit.

Extract Last Digit

The modulus operator returns the remainder. Dividing any integer by 10 leaves its last digit as the remainder.

KEY FORMULA / RULE

$$\text{lastDigit} = n \% 10$$

Java implementation

```
public class LastDigit {  
    public static void main(String[] args) {  
        int n = 735;  
        int lastDigit = n % 10;  
  
        System.out.println(lastDigit); // 5  
    }  
}
```

Worked example

$735 \% 10 = 5$. This operation is the foundation of reverse, palindrome, Armstrong, digit-sum, and digit-frequency problems.

COMPLEXITY

Time: $O(1)$ Space: $O(1)$

COMMON MISTAKE / TIP

For negative numbers, Java's remainder can be negative. If you specifically need a non-negative digit, consider `Math.abs(n % 10)`.

Remove Last Digit

Integer division by 10 discards the fractional part, effectively removing the last decimal digit.

KEY FORMULA / RULE

$$n = n / 10$$

Java implementation

```
public class RemoveLastDigit {  
    public static void main(String[] args) {  
        int n = 12345;  
  
        n = n / 10;  
  
        System.out.println(n); // 1234  
    }  
}
```

Worked example

$12345 / 10 = 1234$ in integer division. The remaining digits are 1234? Careful: $12345 / 10 = 1234.5$. The same operation repeated eventually reaches 0.

COMPLEXITY

One removal: $O(1)$. Repeated digit processing: $O(\log_{10} n)$.

COMMON MISTAKE / TIP

Java integer division truncates the decimal part: $12345 / 10 = 1234$, not 1234.5 .

Reverse a Number

Take the last digit, append it to the result, then remove the last digit from the original number. Repeat until the number becomes 0.

KEY FORMULA / RULE

```
digit = n % 10
rev = rev * 10 + digit
n = n / 10
```

Java implementation

```
public class ReverseNumber {
    static int reverse(int n) {
        int rev = 0;

        while (n > 0) {
            int digit = n % 10;
            rev = rev * 10 + digit;
            n = n / 10;
        }

        return rev;
    }
}
```

Worked example

1234 → digit 4 → rev 4 → digit 3 → rev 43 → digit 2 → rev 432 → digit 1 → rev 4321.

COMPLEXITY

Time: $O(\log_{10} n)$ Space: $O(1)$

COMMON MISTAKE / TIP

If the reversed result may exceed int, use long and, in strict problems, check overflow.

Prime Number

A prime number has exactly two positive factors: 1 and itself. To test primality efficiently, you only need to check possible divisors up to $\sqrt{n}$.

KEY FORMULA / RULE

Check $i = 2 \dots \sqrt{n}$

If $n \% i == 0 \rightarrow$ NOT PRIME

Java implementation

```
public class PrimeNumber {
    static boolean isPrime(int n) {
        if (n < 2) return false;

        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) {
                return false;
            }
        }
        return true;
    }
}
```

Worked example

17 is prime. You only test 2, 3, 4 because $\sqrt{17} \approx 4.12$. None divides 17.

COMPLEXITY

Time: $O(\sqrt{n})$ Space: $O(1)$

COMMON MISTAKE / TIP

1 is not prime. Also, testing all numbers from 2 to $n-1$ is correct but unnecessarily slow.

Count Factors

Factors occur in pairs. If i divides n , then n / i is also a factor. Therefore, iterate only up to $\sqrt{n}$ and add two factors for every pair.

KEY FORMULA / RULE

If $n \% i == 0 \rightarrow$ factors: i and n/i

Java implementation

```
public class CountFactors {
    static int countFactors(int n) {
        int count = 0;

        for (int i = 1; i <= Math.sqrt(n); i++) {
            if (n % i == 0) {
                count += 2;

                if (i == n / i) {
                    count--; // perfect square
                }
            }
        }
        return count;
    }
}
```

Worked example

For 36: (1,36), (2,18), (3,12), (4,9), (6,6) $\rightarrow$ 9 total factors. The pair (6,6) must be counted once.

COMPLEXITY

Time: $O(\sqrt{n})$ Space: $O(1)$

COMMON MISTAKE / TIP

For a perfect square, do not count $\sqrt{n}$ twice.

The Euclidean algorithm repeatedly replaces (a, b) with (b, a % b). When b becomes 0, a is the GCD.

KEY FORMULA / RULE

$\text{gcd}(a, b) = \text{gcd}(b, a \% b)$

Base case: $\text{gcd}(a, 0) = a$

Java implementation

```
public class GCD {  
    static int gcd(int a, int b) {  
        a = Math.abs(a);  
        b = Math.abs(b);  
  
        while (b != 0) {  
            int temp = a % b;  
            a = b;  
            b = temp;  
        }  
        return a;  
    }  
}
```

Worked example

$\text{gcd}(48, 18) \rightarrow \text{gcd}(18, 12) \rightarrow \text{gcd}(12, 6) \rightarrow \text{gcd}(6, 0) \rightarrow 6$.

COMPLEXITY

Time: $O(\log(\min(a, b)))$ Space: $O(1)$ iterative.

COMMON MISTAKE / TIP

Always update both values correctly. The new pair is (b, a % b).

LCM is the smallest positive number divisible by both inputs. The efficient relationship with GCD is used instead of checking multiples one by one.

KEY FORMULA / RULE

$$\text{LCM}(a,b) \times \text{GCD}(a,b) = a \times b$$

$$\text{LCM}(a,b) = |a \times b| / \text{GCD}(a,b)$$

Java implementation

```
public class LCM {  
    static long lcm(long a, long b) {  
        if (a == 0 || b == 0) return 0;  
  
        long g = gcd(a, b);  
        return Math.abs((a / g) * b);  
    }  
  
    static long gcd(long a, long b) {  
        a = Math.abs(a);  
        b = Math.abs(b);  
  
        while (b != 0) {  
            long temp = a % b;  
            a = b;  
            b = temp;  
        }  
        return a;  
    }  
}
```

Worked example

For 12 and 18: GCD = 6. LCM = $(12 / 6) \times 18 = 36$.

COMPLEXITY

GCD: $O(\log \min(a,b))$. LCM uses the GCD, so overall is also logarithmic.

COMMON MISTAKE / TIP

Use $(a / \text{gcd}) * b$ rather than $a * b / \text{gcd}$ when possible to reduce overflow risk.

Armstrong Number

For a k-digit number, add each digit raised to the power k. If the sum equals the original number, it is an Armstrong number.

KEY FORMULA / RULE

$$\text{sum}(\text{digit}^k) = n$$

where k = number of digits

Java implementation

```
public class Armstrong {
    static int power(int base, int exp) {
        int result = 1;
        for (int i = 0; i < exp; i++) {
            result *= base;
        }
        return result;
    }

    static boolean isArmstrong(int n) {
        int original = n;
        int digits = String.valueOf(n).length();
        int sum = 0;

        while (n > 0) {
            int digit = n % 10;
            sum += power(digit, digits);
            n /= 10;
        }

        return sum == original;
    }
}
```

Worked example

$153 \rightarrow 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153 \rightarrow \text{Armstrong.}$

COMPLEXITY

For k digits, roughly $O(k^2)$ with repeated multiplication for power; with k treated as the input size, this is still very small for normal integer ranges.

COMMON MISTAKE / TIP

The exponent is the number of digits, not always 3.

Palindrome Number

A number is a palindrome if it remains the same after reversing it.

KEY FORMULA / RULE

original == reversed → PALINDROME

Java implementation

```
public class Palindrome {  
    static boolean isPalindrome(int n) {  
        if (n < 0) return false;  
  
        int original = n;  
        int rev = 0;  
  
        while (n > 0) {  
            int digit = n % 10;  
            rev = rev * 10 + digit;  
            n /= 10;  
        }  
  
        return original == rev;  
    }  
}
```

Worked example

1221 → reverse = 1221 → palindrome. 123 → reverse = 321 → not palindrome.

COMPLEXITY

Time: $O(\log_{10} n)$ Space: $O(1)$

COMMON MISTAKE / TIP

Save the original number before modifying n. Otherwise you cannot compare the final reverse with the original.

Powers / Exponents

x^n means multiplying x by itself n times. In beginner problems, repeated multiplication is enough; later, fast exponentiation reduces the number of multiplications.

KEY FORMULA / RULE

$$x^1 = x$$

$$x^2 = x \times x$$

$$x^n = x \times x \times \dots \times x \text{ (n times)}$$

Java implementation

```
public class Power {  
    static long power(long x, int n) {  
        long result = 1;  
  
        for (int i = 0; i < n; i++) {  
            result *= x;  
        }  
  
        return result;  
    }  
}
```

Worked example

$$2^5 = 2 \times 2 \times 2 \times 2 \times 2 = 32. \quad 3^3 = 27.$$

COMPLEXITY

Simple loop: $O(n)$ time, $O(1)$ space.

COMMON MISTAKE / TIP

Do not confuse $\wedge$ with exponentiation in Java. In Java, $\wedge$ is bitwise XOR. Use a loop or `Math.pow()`.

Square Root Optimization

Many number problems do not need to inspect every value from 1 to n . Factor pairs guarantee that if a factor exists above $\sqrt{n}$, its partner exists at or below $\sqrt{n}$.

KEY FORMULA / RULE

Naive: $1 \dots n \rightarrow O(n)$

Optimized: $1 \dots \sqrt{n} \rightarrow O(\sqrt{n})$

Java implementation

```
public class SqrtCheck {  
    static boolean hasFactor(int n) {  
        for (int i = 1; i * i <= n; i++) {  
            if (n % i == 0) {  
                return true;  
            }  
        }  
        return false;  
    }  
}
```

Worked example

For $n = 36$, once you inspect factors up to 6, you have covered every factor pair: 1×36 , 2×18 , 3×12 , 4×9 , 6×6 .

COMPLEXITY

Looping to $\sqrt{n}$ changes the complexity from $O(n)$ to $O(\sqrt{n})$.

COMMON MISTAKE / TIP

Prefer $i * i <= n$ or $i <= \text{Math.sqrt}(n)$. With very large long values, $i * i$ can overflow, so use a safer comparison when needed.

Modulus Operator (%)

The modulus operator returns the remainder after integer division. It is one of the most important operators in number-based DSA problems.

KEY FORMULA / RULE

$n \% m \rightarrow$ remainder after dividing n by m

Java implementation

```
public class ModulusDemo {  
    public static void main(String[] args) {  
        System.out.println(10 % 2);    // 0  
        System.out.println(15 % 2);    // 1  
        System.out.println(1234 % 10); // 4  
        System.out.println(17 % 5);    // 2  
    }  
}
```

Worked example

$10 \% 2 = 0 \rightarrow$ even. $15 \% 2 = 1 \rightarrow$ odd. $1234 \% 10 = 4 \rightarrow$ last digit. $17 \% 5 = 2 \rightarrow$ remainder.

COMPLEXITY

A single modulus operation is treated as $O(1)$ for normal machine integer types.

COMMON MISTAKE / TIP

Remember: `%` gives the remainder; `/` gives the quotient.

Logarithms

A logarithm answers: how many times must we multiply the base by itself to reach n ?

KEY FORMULA / RULE

$\log_a(n) = x$ means $a^x = n$

$\log_2(16) = 4$ because $2^4 = 16$

Java implementation

```
public class Logarithm {  
    public static void main(String[] args) {  
        double n = 16;  
        double base = 2;  
  
        double result = Math.log(n) / Math.log(base);  
        System.out.println(result); // 4.0  
    }  
}
```

Worked example

$\log_2(8) = 3$ because $2^3 = 8$. In algorithm analysis, logarithms appear when the problem size is repeatedly divided, such as binary search.

COMPLEXITY

A logarithmic loop often looks like: $n = n / 2$. Number of iterations is $O(\log n)$.

COMMON MISTAKE / TIP

Java's `Math.log()` is the natural logarithm (base e). To get log base b , use `Math.log(n) / Math.log(b)`.

Quick Revision Cheat Sheet

Task	Core operation / idea	Typical complexity
Even / Odd	$n \% 2 == 0$	$O(1)$
Last digit	$n \% 10$	$O(1)$
Remove last digit	$n / 10$	$O(1)$ per digit
Count digits	repeat $n /= 10$	$O(\log n)$
Reverse number	digit = $n \% 10$; rev = $rev * 10 + \text{digit}$	$O(\log n)$
Prime check	test divisors up to $\sqrt{n}$	$O(\sqrt{n})$
Count factors	factor pairs up to $\sqrt{n}$	$O(\sqrt{n})$
GCD	Euclidean algorithm	$O(\log \min(a,b))$
LCM	$(a/\text{gcd}) * b$	$O(\log \min(a,b))$
Palindrome	original == reverse	$O(\log n)$
Armstrong	$\text{sum}(\text{digit}^k) == n$	depends on digits
Power	repeated multiplication	$O(n)$
Modulus	remainder	$O(1)$
Logarithm	reverse of exponentiation	$O(\log n)$ patterns

The 4-digit pattern you should memorize

1. Extract → digit = $n \% 10$
2. Use the digit → sum / compare / build answer
3. Remove → $n = n / 10$
4. Repeat → while ($n > 0$)

This single pattern solves a huge family of beginner Java math problems.

Practice order: Even/Odd → Modulus → Last Digit → Remove Digit → Count Digits → Reverse → Palindrome → Prime → Factors → GCD → LCM → Armstrong → Power → $\sqrt{n}$ optimization → Logarithms.

End of Basics Math • Java Notes